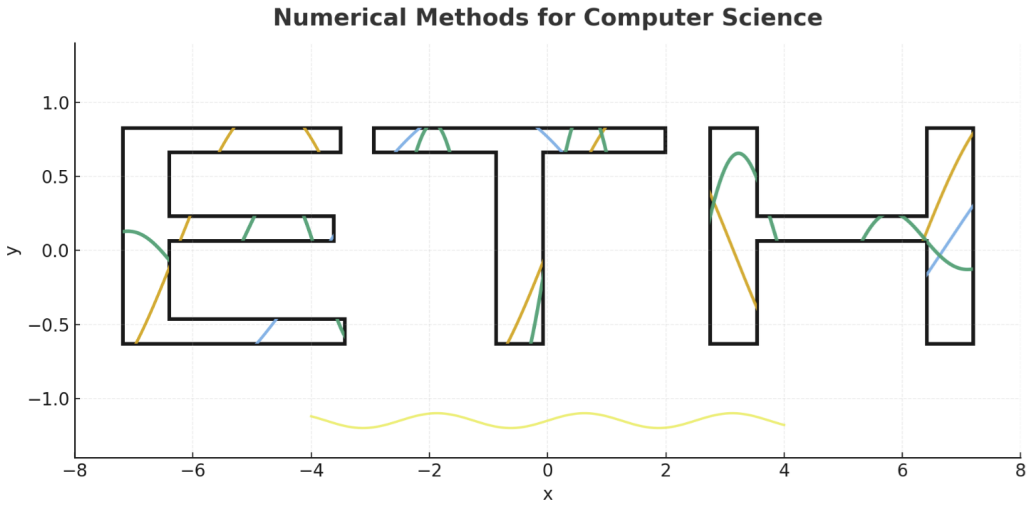


Interpolation — Introduction & Big Picture — Week #3

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This document introduces the core ideas of interpolation, focusing on why, when, and how to use it safely in computation. The following page lists the contents; the body includes concise statements, examples, and code snippets.

Contents

1 What is Interpolation?

We are given a set of distinct nodes (also called *support points*, or *knots*)

$$x_0,\ x_1,\ \ldots,\ x_n \in \mathbb{R}, \qquad x_i \neq x_j \text{ for } i \neq j,$$

together with data values

$$y_i = f(x_i) \in \mathbb{R}, \qquad i = 0, \ldots, n,$$

which are samples of (an unknown or intractable) function f.

Interpolation asks for a function $\tilde{f}$ that agrees with the data at the nodes,

$$\tilde{f}(x_i) = y_i \quad \text{for all } i = 0, \ldots, n,$$

and that can be evaluated cheaply for x between the nodes.

There are *many different ways* to construct such an interpolant $\tilde{f}$:

- **Piecewise linear interpolation:** connect the data points by straight line segments. This method is simple, robust, and produces a continuous but not differentiable interpolant.
- **Spline interpolation:** use piecewise polynomials with continuity conditions at the nodes (e.g. cubic splines), producing a smooth curve well-suited for graphics and geometric modeling.
- **Polynomial interpolation:** construct a single global polynomial $p \in P_n$ of degree at most n that passes exactly through all the data points.

Each of these approaches has its own strengths: piecewise methods are extremely stable and easy to implement, while splines are the standard choice in engineering and computer graphics. Polynomial interpolation, however, is mathematically fundamental: it connects to linear algebra, numerical stability, and approximation theory. It also illustrates many of the challenges (and pitfalls) that arise in numerical analysis.

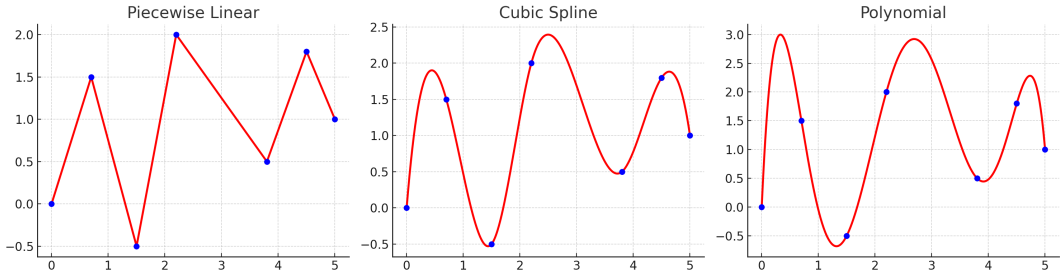


Figure 1: Comparison of interpolation methods: piecewise linear (left), cubic spline (middle), and polynomial interpolation (right). All interpolants pass exactly through the given data points (blue).

In this week, we focus on polynomial interpolation. Our goal is to construct $p \in P_n$, the

vector space of polynomials of degree at most n, that satisfies

$$p(x_i) = y_i, \quad \text{for all } i = 0, \dots, n.$$

In explicit form, such a polynomial looks like

$$p(x) \;=\; a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n,$$

with unknown coefficients $a_0, \dots, a_n$. The interpolation conditions

$$p(x_i) = y_i, \qquad i = 0, \dots, n,$$

translate into a linear system of $n+1$ equations for the $n+1$ unknowns $a_0, \dots, a_n$. Because the system matrix (a Vandermonde matrix) is invertible whenever the x_i are distinct, there always exists a unique interpolating polynomial of degree at most n.

Note

Why do we choose to work with *polynomial* interpolation?

1. **Vector space structure.** The set P_n of all polynomials of degree at most n is an $(n+1)$-dimensional vector space. Writing $p(x) = \sum_{k=0}^{n} a_k x^k$ corresponds exactly to expanding in the monomial basis $\{1, x, x^2, \dots, x^n\}$. The interpolation conditions $p(x_i) = y_i$ translate into a linear system with Vandermonde matrix, making the problem fit naturally into the framework of linear algebra.
2. **Ease of calculus.** Polynomials are the simplest smooth functions: they are easy to differentiate and integrate. Closed-form formulas are straightforward, which is particularly useful in applications involving numerical differentiation, integration, or solving differential equations.
3. **Universality (approximation).** By Taylor's theorem and the Weierstrass approximation theorem, *any* sufficiently regular function can be locally or globally approximated by polynomials. Thus, interpolating polynomials serve as a canonical tool for approximation.
4. **Efficient evaluation.** Polynomials can be evaluated very quickly with only multiplications and additions. In particular, we will soon see Horner's rule, an $O(n)$ algorithm that is both efficient and stable.

For these reasons, polynomial interpolation is not only mathematically elegant but also practically powerful.

1.1 Existence and Uniqueness

Because $\dim P_n = n+1$ and the $n+1$ interpolation conditions $p(x_i) = y_i$ are linearly independent whenever the nodes are distinct, *there exists a unique* polynomial $p \in P_n$ that interpolates the data.

Let us make this concrete. In the monomial basis

$$p(x) \;=\; \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \cdots + \alpha_n x^n,$$

the interpolation conditions

$$p(x_i) = y_i, \qquad i = 0, \ldots, n,$$

translate into the linear system

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix}}_{V \text{ (Vandermonde)}} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

If V is invertible, then the coefficients are uniquely determined as

$$\begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} \;=\; V^{-1} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Note

Why "degree at most n"? Although the system involves all powers up to x^n, it may happen that some $\alpha_k = 0$, in which case the true degree of p is lower than n. The uniqueness guarantee means no other polynomial of smaller degree can interpolate the same $n+1$ points.

Why is V invertible? (linear algebra recap)

The Vandermonde matrix has the form

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix}.$$

We want to show

$$\det(V) = \prod_{0 \le i < j \le n} (x_j - x_i), \qquad V = \left[x_i^j\right]_{i=0,\ldots,n}^{j=0,\ldots,n}.$$

Warm-up: $n = 2$ **(size** 3×3**).** Start from

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 \\ 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \end{bmatrix}, \quad (C_0, C_1, C_2) = \text{columns of degrees } (0, 1, 2).$$

Step 1 (rightmost column): perform $C_2 \leftarrow C_2 - x_0\, C_1$ (this preserves det):

$$V^{(1)} = \begin{bmatrix} 1 & x_0 & x_0^2 - x_0 \cdot x_0 \\ 1 & x_1 & x_1^2 - x_0 \cdot x_1 \\ 1 & x_2 & x_2^2 - x_0 \cdot x_2 \end{bmatrix} = \begin{bmatrix} 1 & x_0 & 0 \\ 1 & x_1 & (x_1 - x_0)\, x_1 \\ 1 & x_2 & (x_2 - x_0)\, x_2 \end{bmatrix}.$$

Step 2 (next column): now $C_1 \leftarrow C_1 - x_0\, C_0$ (again preserves det):

$$V^{(2)} = \begin{bmatrix} 1 & x_0 - x_0 & 0 \\ 1 & x_1 - x_0 & (x_1 - x_0)\, x_1 \\ 1 & x_2 - x_0 & (x_2 - x_0)\, x_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & (x_1 - x_0) & (x_1 - x_0)\, x_1 \\ 1 & (x_2 - x_0) & (x_2 - x_0)\, x_2 \end{bmatrix}.$$

Factorization from rows 2–3: factor $(x_1 - x_0)$ from row 2 and $(x_2 - x_0)$ from row 3:

$$= (x_1 - x_0)(x_2 - x_0) \begin{bmatrix} 1 & 0 & 0 \\ \dfrac{1}{x_1 - x_0} & 1 & x_1 \\ \dfrac{1}{x_2 - x_0} & 1 & x_2 \end{bmatrix}$$

Expand along the first row (or delete row 1/col 1):

$$\det(V) = (x_1 - x_0)(x_2 - x_0) \det \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \end{bmatrix} = (x_1 - x_0)(x_2 - x_0)(x_2 - x_1).$$

This matches $\prod_{0 \le i < j \le 2} (x_j - x_i)$.

General case $(n+1) \times (n+1)$**.** Write columns by degree: $C_0, C_1, \ldots, C_n$ with $C_k = (x_0^k, \ldots, x_n^k)^\top$.

Right-to-left column eliminations: for $k = n, n-1, \ldots, 1$ perform

$$C_k \;\leftarrow\; C_k - x_0\, C_{k-1}.$$

These are elementary column operations that preserve the determinant. After these updates, the matrix becomes

$$\widetilde{V} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & (x_1 - x_0) & (x_1^2 - x_0 x_1) & \cdots & (x_1^n - x_0 x_1^{n-1}) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_0) & (x_n^2 - x_0 x_n) & \cdots & (x_n^n - x_0 x_n^{n-1}) \end{bmatrix}.$$

Row-wise, for every $i \geq 1$ and every $k \geq 1$,

$$x_i^k - x_0 x_i^{k-1} = (x_i - x_0)\, x_i^{k-1},$$

hence we can rewrite $\widetilde{V}$ as

$$\widetilde{V} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & (x_1 - x_0) & (x_1 - x_0)\, x_1 & \cdots & (x_1 - x_0)\, x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_0) & (x_n - x_0)\, x_n & \cdots & (x_n - x_0)\, x_n^{n-1} \end{bmatrix}.$$

Factor from each lower row: factor $(x_i - x_0)$ from row i, $i = 1, \ldots, n$:

$$\det(V) = \Big(\prod_{i=1}^{n} (x_i - x_0)\Big) \det \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ \dfrac{1}{x_1 - x_0} & 1 & x_1 & \cdots & x_1^{n-1} \\ \dfrac{1}{x_2 - x_0} & 1 & x_2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \dfrac{1}{x_n - x_0} & 1 & x_n & \cdots & x_n^{n-1} \end{bmatrix}.$$

Remove the first row/column: expanding along the first row (which is $(1, 0, \ldots, 0)$) gives

$$\det(V) = \Big(\prod_{i=1}^{n} (x_i - x_0)\Big) \det \begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^{n-1} \end{bmatrix} = \Big(\prod_{i=1}^{n} (x_i - x_0)\Big) \det(V_n(x_1, \ldots, x_n)).$$

Induction close: apply the same procedure to $V_n(x_1, \ldots, x_n)$, then to $V_{n-1}(x_2, \ldots, x_n)$, etc. We obtain

$$\det(V) = \Big(\prod_{i=1}^{n} (x_i - x_0)\Big)\Big(\prod_{2 \leq i \leq n} (x_i - x_1)\Big) \cdots (x_n - x_{n-1}) = \prod_{0 \leq i < j \leq n} (x_j - x_i).$$

Since all factors are nonzero when the nodes are distinct, $\det(V) \neq 0$, hence V is invertible.

1.2 Three Equivalent Views (glimpse only; details later)

At this point we have already established existence and uniqueness: given $n+1$ points, there is exactly one polynomial p of degree at most n that interpolates them.

If we were not concerned with *efficiency* or *numerical stability*, the story of polynomial interpolation could essentially end here. We would just write down the Vandermonde system, solve for the coefficients, and we would be done.

However, solving a general linear system of size $(n + 1) \times (n + 1)$ costs on the order of $O(n^3)$ operations. Moreover, the Vandermonde matrix quickly becomes ill-conditioned (i.e. very sensitive to small perturbations in the data: tiny changes in x_i or y_i can cause large errors in

the computed coefficients) as n grows. Even if the method is simple in theory, in practice it can be unstable. This motivates alternative representations of the interpolating polynomial, each with its own computational and conceptual advantages.

- **Monomial / Vandermonde form.** Conceptually simplest: write

$$p(x) = \sum_{k=0}^{n} \alpha_k x^k,$$

  and determine α by solving $V\alpha = y$. Advantage: very clear mathematically. Disadvantage: the Vandermonde system is ill-conditioned for moderate n, and the solution costs $O(n^3)$.

- **Newton form (divided differences).** Build p incrementally using the basis

$$1,\ (x - x_0),\ (x - x_0)(x - x_1),\ \ldots$$

  with coefficients given by divided differences. Advantage: efficient to *update* if a new interpolation point arrives; cheaper to evaluate. Disadvantage: requires a recursive computation of divided differences.

- **Lagrange / barycentric form.** Write

$$p(x) = \sum_{i=0}^{n} y_i \, \ell_i(x),$$

  where the ℓ_i are the Lagrange basis polynomials. Advantage: this form makes interpolation conceptually transparent ("the interpolant is a linear combination of basis functions that interpolate Kronecker deltas"). Moreover, the *barycentric* reformulation leads to numerically stable $O(n)$ evaluation once weights are precomputed. Disadvantage: not as convenient as Newton form when inserting new nodes.

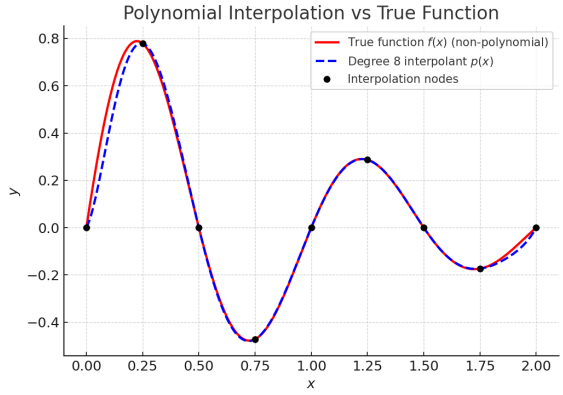
In short: there are three equivalent ways to represent the same interpolating polynomial. All produce the *same* p, but each has different computational properties. In the following sections we will explore them in detail.

Note

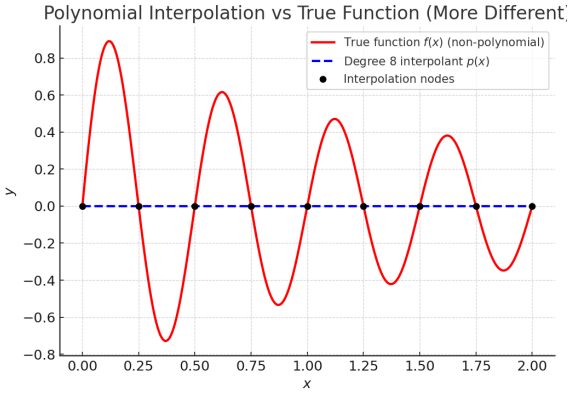
Although we have seen that polynomial interpolation (regardless of the chosen method) always leads to the *same unique polynomial* of degree at most n passing through the $n+1$ data points, it is important to remember a subtle but crucial fact:

There are infinitely many other functions that could also pass through these points. The original f that generated the data could have been anything — smooth or oscillatory, polynomial or not. By focusing on polynomial interpolation, we are *choosing* to approximate f by a polynomial constrained to match the data at the given nodes.

In short: interpolation does not reveal the hidden f, it only produces one possible approximant — the unique polynomial interpolant of degree $\leq n$ through the samples, which, in most practical cases, is sufficiently accurate to simulate or approximate the unknown function.



(a) True function $f(x) = \sin(2\pi x)e^{-x}$ vs. degree 8 interpolant.



(b) True function $f(x) = \frac{\sin(4\pi x)}{1+x}$ vs. degree 8 interpolant.

Figure 2: Polynomial interpolation vs. non-polynomial true functions. In both cases the interpolant matches the data points exactly, but the global behavior can differ significantly.

1.3 Example: Ill-conditioning of the Vandermonde System

Consider interpolation at three very close nodes:

$$x_0 = 1.0, \quad x_1 = 1.0001, \quad x_2 = 1.0002, \qquad y_i = e^{x_i}.$$

The corresponding Vandermonde matrix is

$$V = \begin{bmatrix} 1 & 1.0 & 1.0^2 \\ 1 & 1.0001 & (1.0001)^2 \\ 1 & 1.0002 & (1.0002)^2 \end{bmatrix}.$$

Its determinant evaluates to a very small number (on the order of 10^{-12}), which means that V is nearly singular (non-invertible). Solving for α in $V\alpha = y$ gives coefficients with very large magnitudes (e.g. $\alpha_0 \approx 1.6 \times 10^5$, $\alpha_1 \approx -3.2 \times 10^6$, $\alpha_2 \approx 1.6 \times 10^6$).

Note

This is a hallmark of an **ill-conditioned** system: tiny perturbations in the inputs (for instance due to floating-point rounding) cause disproportionately large changes in the solution. Although the interpolant still passes through the data points, its coefficients are unreliable and its evaluation outside the nodes may be very inaccurate.

2 Efficient Evaluation: Horner's Rule

From a Naive Sum of Powers to Horner's Rule Suppose we have

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n.$$

The first time you try to compute $p(x)$, you probably do it *naively*: for each term $a_k x^k$, you build x^k by multiplying x with itself k times, then you add all the terms together.

Cost of the naive approach. If you form powers sequentially ($x,\ x^2 = x \cdot x,\ x^3 = x^2 \cdot x, \dots$), you perform roughly

$$1 + 2 + \cdots + n \ = \ \frac{n(n+1)}{2}$$

multiplications just to construct the powers, plus n additions to sum them up. Overall, the cost is $O(n^2)$.

The "aha!" moment (factorize the x out). Observe that the same polynomial can be written in a nested way:

$$\begin{aligned} p(x) &= a_0 + x(a_1 + x(a_2 + \cdots + x a_n)\cdots) \\ &= (((a_n x + a_{n-1})x + a_{n-2})x + \cdots)x + a_0. \end{aligned}$$

Define the backward recurrence

$$b_n = a_n, \qquad b_k = a_k + x\, b_{k+1} \quad (k = n-1, \dots, 0),$$

then $p(x) = b_0$. Each step requires exactly one multiplication and one addition. Total: n multiplications + n additions $\Rightarrow O(n)$.

> Note
>
> Horner's rule does not change the polynomial—it only changes the order of arithmetic. You get the same result, but with far fewer operations and typically with better numerical stability (why? -> less space for errors).

Python (ready to use).

```python
def horner(a, x):
    """
    Evaluate p(x) = a[0] + a[1]*x + ... + a[n]*x**n
    using Horner's rule in O(n) time.
    """
    b = a[-1]           # start with a_n
    for k in range(len(a)-2, -1, -1):
        b = a[k] + x*b # one multiply, one add
    return b
```

Optional: value and derivative "for free". With one extra accumulator, Horner's rule also gives $p'(x)$ alongside $p(x)$ in $O(n)$:

Listing 1: Horner for both p(x) and p'(x)

```
def horner_val_der(a, x):
    """
    Returns (p(x), p'(x)) for p with coefficients a[0..n].
    """
    b = a[-1]    # b_n
    c = 0.0      # c_n
    for k in range(len(a)-2, -1, -1):
        c = b + x*c       # c_k = b_{k+1} + x*c_{k+1}
        b = a[k] + x*b    # b_k = a_k + x*b_{k+1}
    return b, c
```

Takeaway. Naive evaluation: $O(n^2)$. Horner's rule: $O(n)$, simple, fast, and the gold standard for polynomial evaluation.

3 Bases for the Polynomial Vector Space

3.1 Introduction

We return to the central problem of interpolation: given $n+1$ distinct nodes

$$x_0, x_1, \ldots, x_n \in \mathbb{R}, \qquad x_i \neq x_j \;\; (i \neq j),$$

together with data values

$$y_0, y_1, \ldots, y_n \in \mathbb{R},$$

we seek a polynomial $p \in P_n$ such that

$$p(x_j) = y_j, \qquad j = 0, \ldots, n.$$

Here P_n denotes the set of all polynomials of degree at most n,

$$P_n := \{\, p(x) = a_0 + a_1 x + \cdots + a_n x^n : a_j \in \mathbb{R} \,\}.$$

It is a vector space of dimension $\dim P_n = n+1$. Thus, specifying $n+1$ independent conditions uniquely determines a polynomial in P_n.

> Note
>
> This vector-space viewpoint explains *why uniqueness holds*. Any choice of basis for P_n (monomials, Newton, Lagrange, etc.) spans the same space, and the interpolating polynomial will be the same—only its representation changes.

3.2 Monomial basis (Vandermonde form)

We have already discussed this representation earlier: $p(x)$ is expressed in the monomial basis $\{1, x, x^2, \ldots, x^n\}$, and the interpolation conditions lead to a linear system with the Vandermonde matrix. This form is conceptually simple but numerically unstable for large n.

3.3 Lagrange basis

Instead of solving a linear system with the Vandermonde matrix, we can directly construct a convenient basis of P_n adapted to the nodes:

$$L_i(x) := \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \qquad i = 0, \ldots, n.$$

$$L_i(x) \;=\; \frac{(x-x_0)}{(x_i-x_0)} \cdot \frac{(x-x_1)}{(x_i-x_1)} \cdots \frac{(x-x_{i-1})}{(x_i-x_{i-1})} \cdot \frac{(x-x_{i+1})}{(x_i-x_{i+1})} \cdots \frac{(x-x_n)}{(x_i-x_n)}.$$

These are the *Lagrange basis polynomials*, satisfying the cardinal property

$$L_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Why does this hold?

- If $x = x_i$, then in every factor we have $(x_i - x_j)/(x_i - x_j) = 1$ for all $j \neq i$. Hence the entire product equals 1.
- If $x = x_j$ with $j \neq i$, then one of the factors becomes $(x_j - x_j)/(x_i - x_j) = 0$, so the whole product vanishes.

Thus, each $L_i(x)$ is designed to "select" node x_i: it takes the value 1 at x_i and 0 at all other nodes.

Any interpolant can then be written directly as

$$p(x) = \sum_{i=0}^{n} y_i \, L_i(x).$$

Indeed, by construction we have

$$p(x_j) = \sum_{i=0}^{n} y_i \, L_i(x_j) = y_j,$$

since $L_i(x_j) = \delta_{ij}$. Thus this polynomial interpolates the data exactly.

Moreover, we know from the dimension argument that for a given set of $n+1$ distinct nodes, there exists a *unique* polynomial of degree at most n satisfying $p(x_i) = y_i$. Since the expression above is such a polynomial, we conclude that

$$p(x) = \sum_{i=0}^{n} y_i \, L_i(x)$$

is precisely the unique interpolating polynomial.

3.4 Newton basis

Another natural choice of basis for P_n is the *Newton basis* associated with the interpolation nodes. For a given set of distinct nodes $x_0, x_1, \ldots, x_n$, define

$$N_0(x) := 1, \qquad N_1(x) := (x - x_0), \qquad N_2(x) := (x - x_0)(x - x_1), \quad \ldots, \quad N_k(x) := \prod_{j=0}^{k-1} (x - x_j).$$

Thus, in general

$$N_k(x) \;=\; (x - x_0)(x - x_1) \cdots (x - x_{k-1}), \qquad k = 0, 1, \ldots, n.$$

Clearly $\{N_0, N_1, \ldots, N_n\}$ forms a basis of P_n.

Why is this a basis?

- Each $N_k(x)$ is a polynomial of degree exactly k.
- Therefore $N_0, N_1, \ldots, N_n$ are linearly independent: no lower-degree combination can reproduce a higher-degree term.
- The space P_n has dimension $n+1$, and we have exactly $n+1$ such polynomials.

Hence the Newton polynomials span P_n.

> Note
>
> Another way to think about this: each new factor $(x - x_j)$ "forces" the degree to increase by one, contributing a genuinely new dimension to the space. Thus the Newton basis grows step by step with the degree.

Newton form of the interpolant Any polynomial $p \in P_n$ can now be written as

$$p(x) \;=\; c_0 N_0(x) + c_1 N_1(x) + \cdots + c_n N_n(x).$$

The interpolation conditions $p(x_i) = y_i$ lead to a linear system for the coefficients $c_0, \ldots, c_n$. But this system is *triangular*, unlike the Vandermonde case!

Why triangular? Let us check the structure explicitly:

- At $x = x_0$:

$$p(x_0) = c_0 N_0(x_0) = c_0 = y_0.$$

So $c_0 = y_0$ directly.

- At $x = x_1$:

$$p(x_1) = c_0 N_0(x_1) + c_1 N_1(x_1) = c_0 + c_1(x_1 - x_0) = y_1.$$

- At $x = x_2$:

$$p(x_2) = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) = y_2.$$

And so on. At each stage $x = x_k$, the only new term that survives is $c_k N_k(x_k)$, while higher terms vanish because $N_m(x_k) = 0$ for all $m > k$ (since one factor in the product $(x_k - x_k)$ appears).

Therefore the interpolation matrix in Newton basis is lower triangular:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \cdots \\ 1 & (x_1 - x_0) & 0 & 0 & \cdots \\ 1 & (x_2 - x_0) & (x_2 - x_0)(x_2 - x_1) & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_0) & (x_n - x_0)(x_n - x_1) & \cdots & (x_n - x_0)\cdots(x_n - x_{n-1}) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Key advantage Unlike the Vandermonde system (dense and ill-conditioned), this matrix is *lower triangular*, so the coefficients c_k can be computed sequentially by forward substitution. Even better, these c_k coefficients turn out to be exactly the *divided differences* of the data — which we will derive next.

Note

The Newton basis makes the interpolation problem incremental: when a new node (x_{n+1}, y_{n+1}) is added, we do not need to recompute all coefficients from scratch. Only the last coefficient c_{n+1} must be updated.

Efficient evaluation (Horner-like scheme). Once the coefficients $c_0, \ldots, c_n$ are known, the Newton form

$$p(x) \;=\; c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

can be evaluated naively in $O(n^2)$ operations (because of all repeated products). However, we can regroup terms recursively, exactly like Horner's rule:

$$p(x) = c_0 + (x - x_0)\Big(c_1 + (x - x_1)(c_2 + \cdots + (x - x_{n-1})c_n)\Big).$$

This nested structure reduces the cost to only n multiplications and n additions, i.e. $O(n)$ arithmetic overall.

Note

This is sometimes called the *generalized Horner scheme* for Newton polynomials. It shows that the Newton form is not only convenient for constructing the interpolant incrementally, but also for evaluating it very efficiently at any x.

4 Algorithms for Polynomial Interpolation

4.1 Motivation

The interpolation problem is conceptually simple: given $n+1$ distinct nodes (x_i, y_i), there exists a *unique* polynomial $p \in P_n$ such that

$$p(x_i) = y_i, \qquad i = 0, \dots, n.$$

However, there are many different ways to *compute and evaluate* this polynomial in practice. All representations yield the same interpolant, but the computational cost and numerical stability depend strongly on the chosen algorithm.

4.2 Different computational contexts

When choosing an algorithm, it is essential to clarify the context:

- **Single evaluation:** we need $p(x^*)$ at one new point x^* only. The priority is minimal one-time cost.
- **Multiple evaluations:** we want $p(x)$ at many points (e.g. for plotting, or in numerical integration). Here precomputations may pay off if they reduce per-point cost.
- **Incremental interpolation:** sometimes new data points arrive one by one. Ideally, we should update the interpolant efficiently without recomputing everything.
- **Extrapolation:** evaluating $p(x)$ outside the range of the given nodes. This is often unstable, and the algorithm's form strongly affects accuracy.

Thus, while the mathematical interpolant is unique, the *algorithmic path* to it depends on practical considerations of cost and stability.

Note

The three most common algorithmic viewpoints are:

1. Monomial basis (Vandermonde system),
2. Newton form (divided differences),
3. Lagrange form (with barycentric evaluation).

Each has its strengths and weaknesses in different contexts.

4.3 Multiple evaluation

The task. We fix once and for all the interpolation *nodes*

$$t_0,\ t_1,\ \dots,\ t_n,$$

and we will face many different *data sets*

$$(y_0, y_1, \ldots, y_n).$$

For each new data set, we want to construct the interpolant and evaluate it efficiently at many target points $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$.

Key difficulty: a naive approach (solving a Vandermonde system for every new y and then evaluating at all $x^{(k)}$) would be prohibitively expensive. We need a representation that reuses precomputations tied only to the nodes $t_0, \ldots, t_n$.

> Note
>
> This is the scenario where the Lagrange/barycentric formula is particularly effective: the structure depends only on the nodes, while the data values y_i enter linearly.

Python skeleton (class interface). We can encapsulate this in a class that stores the fixed nodes at construction and provides an `eval` method for different y-vectors and target x-points:

Listing 2: Skeleton for polynomial interpolation with fixed nodes

```
import numpy as np

class PolyInterp:
    def __init__(self, t):
        self.t = np.asarray(t, float)
        self.n = len(self.t) - 1
        # precompute barycentric weights later

    def eval(self, y, x):
        y = np.asarray(y, float)
        x = np.asarray(x, float)
        # implement barycentric formula here
        raise NotImplementedError
```

Multiple evaluation (naïve Lagrange form — recall and cost)

Notation. Fix the interpolation nodes

$$\mathbf{t} = (t_0, \ldots, t_n), \qquad \text{and data } \mathbf{y} = (y_0, \ldots, y_n).$$

We want the interpolant evaluated at many targets

$$\mathbf{x} = (x^{(1)}, \ldots, x^{(N)}).$$

Lagrange basis and interpolant (recall). For each $i = 0, \ldots, n$,

$$L_i(x; \mathbf{t}) \;=\; \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - t_j}{t_i - t_j}, \qquad \text{and} \qquad p(x; \mathbf{t}, \mathbf{y}) \;=\; \sum_{i=0}^{n} y_i \, L_i(x; \mathbf{t}).$$

Naïve cost for many x (before optimizations). Consider evaluating $p(x)$ at a single target x:

- For a fixed index i, computing $L_i(x; \mathbf{t})$ requires a product over all $j \neq i$: this is $\Theta(n)$ multiplications (plus a division by the constant denominator).
- Doing this for all $i = 0, \ldots, n$ costs $\Theta(n) \times (n + 1) = \Theta(n^2)$ flops.
- The final weighted sum $\sum_i y_i L_i(x; \mathbf{t})$ adds only $O(n)$ more operations, which is dominated by $\Theta(n^2)$.

Therefore, one evaluation $x \mapsto p(x)$ costs $\Theta(n^2)$ in the naïve Lagrange product form. For N target points $x^{(1)}, \ldots, x^{(N)}$, the total cost is

$$\boxed{\Theta(n^2 \, N)}.$$

Note

Even if you precompute the denominators $\prod_{j \neq i}(t_i - t_j)$ once (a one-time $O(n^2)$ setup that depends only on the nodes), the per-target cost remains $\Theta(n^2)$ because the numerators $\prod_{j \neq i}(x - t_j)$ must still be rebuilt for every new x. In the next step (barycentric form) we will reduce the per-target cost to $O(n)$.

4.4 Barycentric form of the interpolant

Motivation. In the naïve Lagrange formula, each basis polynomial $L_i(x; \mathbf{t})$ requires $O(n)$ multiplications, giving a total cost $O(n^2)$ per evaluation. We now reorganize the formula so that evaluation becomes only $O(n)$.

Step 1: introduce barycentric weights. Define for each node t_i the weight

$$w_i \;:=\; \frac{1}{\prod_{\substack{j=0 \\ j \neq i}}^{n} (t_i - t_j)}.$$

These w_i depend only on the nodes $\mathbf{t}$ (not on y or x). They can be precomputed once in $O(n^2)$ time.

Step 2: rewrite the Lagrange basis. Recall

$$L_i(x;\mathbf{t}) = \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x-t_j}{t_i-t_j}.$$

Factor the denominator into w_i^{-1}:

$$L_i(x;\mathbf{t}) = \left(\prod_{\substack{j=0 \\ j\neq i}}^{n} (x-t_j)\right) \cdot w_i.$$

Step 3: collect common factors. Let

$$Q(x) := \prod_{j=0}^{n} (x-t_j).$$

Then for each i,

$$L_i(x;\mathbf{t}) = \frac{Q(x)}{x-t_i}\, w_i.$$

Step 4: assemble the interpolant.

$$p(x;\mathbf{t},\mathbf{y}) = \sum_{i=0}^{n} y_i L_i(x;\mathbf{t}) = Q(x) \sum_{i=0}^{n} \frac{w_i}{x-t_i}\, y_i.$$

Step 5: cancel the common factor. Note that

$$\sum_{i=0}^{n} L_i(x;\mathbf{t}) = 1 \quad \Rightarrow \quad 1 = Q(x) \sum_{i=0}^{n} \frac{w_i}{x-t_i}.$$

Therefore,

$$p(x;\mathbf{t},\mathbf{y}) = \frac{\displaystyle\sum_{i=0}^{n} \frac{w_i}{x-t_i}\, y_i}{\displaystyle\sum_{i=0}^{n} \frac{w_i}{x-t_i}}.$$

Final barycentric formula.

$$\boxed{p(x;\mathbf{t},\mathbf{y}) = \frac{\sum_{i=0}^{n} \frac{w_i}{x-t_i}\, y_i}{\sum_{i=0}^{n} \frac{w_i}{x-t_i}}.}$$

Special case at the nodes. If $x = t_j$ exactly, the formula has removable singularities. In this case we define

$$p(t_j;\mathbf{t},\mathbf{y}) := y_j,$$

which matches the interpolation condition.

Note

Mathematically, the two forms

$$p(x) = Q(x) \sum_{i=0}^{n} \frac{w_i}{x - t_i} y_i \quad \text{and} \quad p(x) = \frac{\sum_{i=0}^{n} \frac{w_i}{x-t_i} y_i}{\sum_{i=0}^{n} \frac{w_i}{x-t_i}}$$

are *identical*. The second form simply cancels the common factor $Q(x)$. However, in practice this cancellation is crucial:

- Evaluating $Q(x)$ directly can be numerically unstable, since it may become very large or very small when x lies near a node.
- The barycentric form avoids this instability by never forming $Q(x)$ explicitly.
- At the same time, the cost per evaluation drops from constructing products to just computing two sums of length $n + 1$.

Thus, although both expressions are mathematically equivalent, the barycentric form is vastly superior for numerical computation.

Cost analysis.

- Precompute w_i once: $O(n^2)$ operations.
- Each evaluation $x \mapsto p(x)$ requires two sums of $n + 1$ terms: $O(n)$.
- For $N >> n$ target points, total cost is $O(nN)$.

Note

This is an enormous improvement over the naïve $O(n^2 N)$ cost. In fact, the barycentric form is the standard algorithm in practice: precompute once, then evaluate very quickly and stably at any number of points.

4.5 Single evaluation (Aitken–Neville / Neville's algorithm)

Task. Given distinct nodes $\mathbf{t} = (t_0, \ldots, t_n)$ and data $\mathbf{y} = (y_0, \ldots, y_n)$, evaluate the Lagrange interpolant $p \in P_n$ at a single target $x \in \mathbb{R}$:

$$\text{compute} \quad p(x) \quad \text{once, with minimal setup.}$$

Partial interpolants. For $0 \le k \le \ell \le n$ let $p_{k,\ell}(x)$ denote the unique polynomial of degree $\le \ell - k$ that interpolates the points (t_j, y_j) for $j = k, \ldots, \ell$.

Base case:

$$p_{k,k}(x) \equiv y_k, \qquad 0 \le k \le n.$$

So each diagonal entry of the tableau starts with the data values themselves.

Neville recurrence. For $k < \ell$, uniqueness tells us that $p_{k,\ell}$ can be expressed as a convex combination of the two smaller interpolants that agree at the endpoints:

$$\boxed{p_{k,\ell}(x) = \frac{(x - t_k)\, p_{k+1,\ell}(x) - (x - t_\ell)\, p_{k,\ell-1}(x)}{t_\ell - t_k}, \qquad 0 \le k < \ell \le n.}$$

Why is this correct?

- The RHS is a polynomial of degree at most $\ell - k$.
- At $x = t_k$ the numerator collapses to $0 - (x - t_\ell)y_k$, so the whole fraction reduces to y_k.
- At $x = t_\ell$ it reduces to y_ℓ.
- For each intermediate node t_j, with $k < j < \ell$, both $p_{k+1,\ell}(t_j)$ and $p_{k,\ell-1}(t_j)$ equal y_j, so the RHS equals y_j.

Thus the RHS interpolates all $\ell - k + 1$ nodes $\{t_k, \ldots, t_\ell\}$, and by uniqueness it must be $p_{k,\ell}(x)$.

Triangular tableau. The recurrence can be visualized as filling a triangular array:

$\ell - k =$	0	1	2	3
t_0	$y_0 =: p_{0,0}(x)$	$\nearrow p_{0,1}(x)$	$\nearrow p_{0,2}(x)$	$\nearrow p_{0,3}(x)$
t_1	$y_1 =: p_{1,1}(x)$	$\nearrow p_{1,2}(x)$	$\nearrow p_{1,3}(x)$	
t_2	$y_2 =: p_{2,2}(x)$	$\nearrow p_{2,3}(x)$		
t_3	$y_3 =: p_{3,3}(x)$			

Figure 3: Neville's algorithm builds the interpolant via a triangular tableau.

At the bottom-left we have the data values y_k. Each new column is obtained from the one to its right using the recurrence. The final result $p(x) = p_{0,n}(x)$ sits at the top-left corner.

In-place $O(n)$ memory. Notice that to compute $p_{k,\ell}$ we only need values from the previous column. Thus we can overwrite a single vector $\mathbf{y}$ of length $n + 1$ as we go, reducing memory from $O(n^2)$ to $O(n)$.

Edge case (node hit). If $x = t_j$ for some j, we immediately return $p(x) = y_j$ to avoid unnecessary computation (and division by zero in the recurrence).

Listing 3: Neville single-point evaluation (in-place, $O(n^2)$ flops, $O(n)$ memory)

Algorithm (Python pseudo-code).

```
import numpy as np

def neville_single_eval(t, y, x):
    t = np.asarray(t, float)
    y = np.asarray(y, float).copy()     # work in-place
    # exact node? return immediately
```

```
    j = np.where(np.isclose(x, t))[0]
    if j.size:
        return float(y[j[0]])
    n = len(t) - 1
    for m in range(1, n+1):              # column width
        for k in range(0, n-m+1):        # row
            i = k + m
            y[k] = ((x - t[k]) * y[k+1] - (x - t[i]) * y[k]) / (t[i] -
                t[k])
    return float(y[0])                   # equals p(x)
```

Cost and when to use. Neville's algorithm costs $O(n^2)$ flops and $O(n)$ memory, with no preprocessing. It is therefore ideal for *single-point evaluation* when:

- the nodes **t** may change between queries,
- or when only one evaluation is needed and precomputation (like barycentric weights) would be wasted.

For many evaluations with fixed nodes, the barycentric formula is superior ($O(n^2)$ setup, then $O(n)$ per evaluation).

> Note
>
> Compared to the Vandermonde approach:
>
> - Vandermonde requires solving $V\alpha = y$ once at $O(n^3)$ cost,
> - then Horner evaluation for each x costs $O(n)$,
> - while Neville avoids the $O(n^3)$ setup and delivers $O(n^2)$ cost for one evaluation.
>
> Thus Neville is the method of choice for *a single target point*, whereas barycentric shines when many evaluations are needed.

Incremental update (Aitken–Neville). When a new point (t_n, y_n) is added to existing nodes $\{(t_0, y_0), \ldots, (t_{n-1}, y_{n-1})\}$, the Aitken–Neville tableau needs only the *new diagonal* to be filled:

$$p_{n-1,n}(x),\ p_{n-2,n}(x),\ \ldots,\ p_{0,n}(x),$$

each computed from

$$p_{k,n}(x) \;=\; \frac{(x - t_k)\, p_{k+1,n}(x) - (x - t_n)\, p_{k,n-1}(x)}{t_n - t_k}, \qquad k = n-1, \ldots, 0.$$

Thus the extra work is $O(n)$ rather than recomputing the whole $O(n^2)$ tableau.

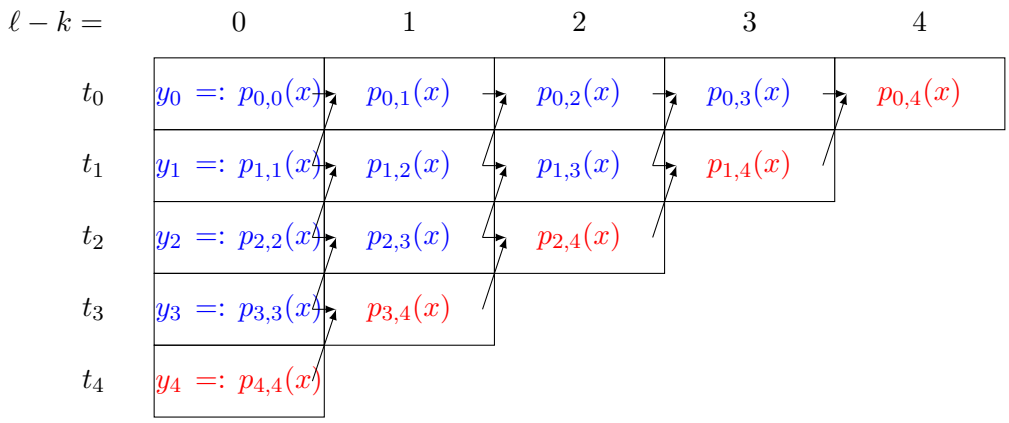


Figure 4: Neville tableau for $t_0, \ldots, t_4$; the new diagonal $\ell = 4$ is highlighted in red. Each $p_{k,\ell}$ uses $p_{k,\ell-1}$ (left) and $p_{k+1,\ell}$ (down-right).

5 Extrapolation to Zero

Extrapolation means interpolation with an evaluation point t outside the interval of nodes. Here we focus on the special case where the target is

$$t = 0, \qquad t_j > 0.$$

In this setting, the task is to compute a limit of the form

$$\lim_{h \to 0} \psi(h),$$

with prescribed accuracy. Direct evaluation of $\psi(h)$ for very small arguments $|h| \ll 1$ is often numerically unstable or even impossible. Therefore, we rely on *Lagrangian extrapolation* instead.

> Note
>
> In practice $\psi(h)$ may only be available through a formula or a black-box routine. Extrapolation provides a systematic way to approximate $\lim_{h \to 0} \psi(h)$ without ever evaluating ψ at dangerously small values of h.

Task. We want to approximate a limit of the form

$$\lim_{h \to 0} \varphi(h),$$

where $\varphi(h)$ can be evaluated for moderately small h, but direct computation at very small h is unstable or impossible.

Key difficulty: for tiny h, numerical cancellation and rounding errors destroy accuracy.

Basic idea.

1. Pick a decreasing sequence of step sizes $h_0 > h_1 > \cdots > h_n > 0$ where $\varphi(h_j)$ is numerically safe to evaluate.
2. Compute the values $\varphi(h_j)$.
3. Interpolate them by a polynomial $p \in P_n$:

$$p(h_j) = \varphi(h_j), \quad j = 0, \ldots, n.$$

4. Approximate the inaccessible limit by

$$\lim_{h \to 0} \varphi(h) \;\approx\; p(0).$$

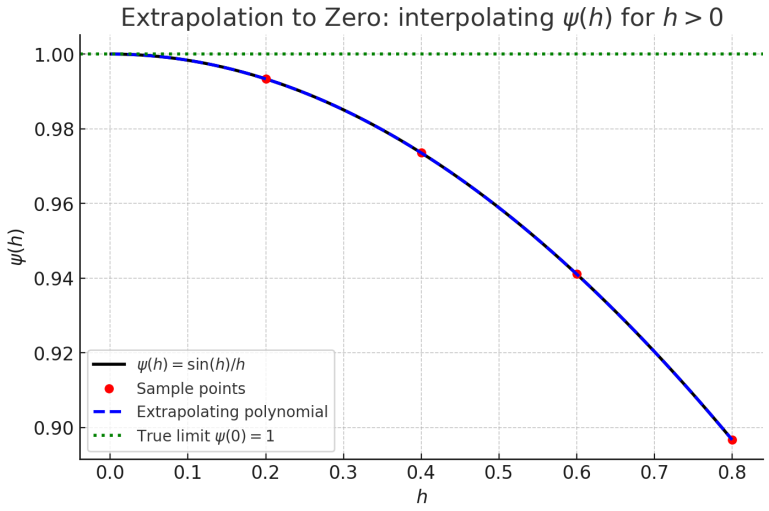


Figure 5: Extrapolation to zero: instead of evaluating $\varphi(h)$ at extremely small h, we interpolate safe values at larger h_j and then evaluate the interpolant at $h = 0$.

Why extrapolation works. The key observation is that for many smooth functions $\psi(h)$ of a small parameter h, a Taylor expansion around $h = 0$ exists:

$$\psi(h) \;=\; A_0 + A_1 h + A_2 h^2 + \cdots + A_m h^m + \mathcal{O}(h^{m+1}).$$

Thus $\psi(h)$ is well-approximated near zero by a polynomial in h. The constant term of this expansion is precisely the desired limit

$$\lim_{h \to 0} \psi(h) = A_0.$$

Extrapolation principle. If we can evaluate $\psi(h)$ at moderately small values $h_0, h_1, \ldots, h_n > 0$, then we may interpolate these values with a polynomial $p(h)$ and use

$$p(0) \approx A_0 = \lim_{h \to 0} \psi(h).$$

The interpolation cancels low-order error terms of the Taylor series systematically, so the

extrapolated value $p(0)$ converges to the true limit with high accuracy, without ever evaluating ψ at dangerously small h.

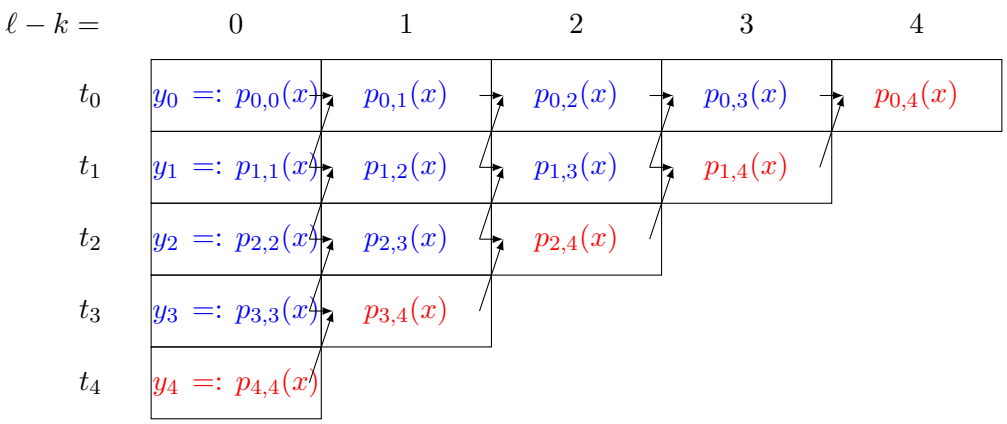
Algorithmic viewpoint. To compute the extrapolated value $p(0)$ we can use Neville's algorithm, which is particularly suitable for this task because it is *update-friendly*: when a new data point $(h_j, \psi(h_j))$ is added, we only need $O(j)$ additional work to update the tableau.

Practical strategy.

1. Start with a few safe step sizes $h_0, h_1, \ldots, h_m$ and compute $\psi(h_j)$.
2. Use Neville's algorithm to evaluate $p_{0,m}(0)$, the interpolant at 0.
3. Check the *quality of approximation*: compare two extrapolated values that use different subsets of the data, e.g.
$$p_{0,m}(0) \quad \text{and} \quad p_{1,m}(0).$$
If these agree within a prescribed tolerance, accept the result.
4. If the discrepancy is too large, add another point h_{m+1} and repeat.

Note

This adaptive procedure is widely used in practice: the algorithm improves the approximation step by step, and stops automatically once the extrapolated values stabilize.



6 Divided differences

Recall: Newton basis. We have seen that the Newton basis

$$N_0(x) = 1, \qquad N_1(x) = (x - t_0), \qquad N_2(x) = (x - t_0)(x - t_1), \quad \ldots, \quad N_j(x) = \prod_{m=0}^{j-1} (x - t_m)$$

forms a basis of P_n, with $\deg N_j = j$ and leading coefficient 1. Therefore, if

$$p(x) = a_0 N_0(x) + a_1 N_1(x) + \cdots + a_n N_n(x),$$

then the last coefficient a_j is always the leading coefficient of p.

Notation. It is convenient to write

$$a_j = y[t_0, \ldots, t_j],$$

the *divided difference* of the data. More generally, for indices $i \leq i + k$ we define

$$y[t_i] := y_i, \qquad y[t_i, \ldots, t_{i+k}] := \frac{y[t_{i+1}, \ldots, t_{i+k}] - y[t_i, \ldots, t_{i+k-1}]}{t_{i+k} - t_i}.$$

Recursive structure. Thus divided differences can be computed step by step: - order 0: $y[t_i] = y_i$, - order 1: $y[t_i, t_{i+1}] = \frac{y_{i+1} - y_i}{t_{i+1} - t_i}$, - order 2: $y[t_i, t_{i+1}, t_{i+2}] = \frac{y[t_{i+1}, t_{i+2}] - y[t_i, t_{i+1}]}{t_{i+2} - t_i}$, and so forth.

Tabular scheme. Just like the Aitken–Neville scheme, divided differences can be arranged in a triangular tableau:

	0	1	2	3
t_0	$y[t_0]$	$y[t_0, t_1]$	$y[t_0, t_1, t_2]$	$y[t_0, t_1, t_2, t_3]$
t_1	$y[t_1]$	$y[t_1, t_2]$	$y[t_1, t_2, t_3]$	
t_2	$y[t_2]$	$y[t_2, t_3]$		
t_3	$y[t_3]$			

Algorithmic cost. The computation requires two nested loops, one for the order of divided differences and one for the position. Hence the total cost is $O(n^2)$ arithmetic, similar to Neville's scheme.

Incremental property. A key advantage: when a new node (t_{n+1}, y_{n+1}) is added, all previous divided differences $y[t_0, \ldots, t_j]$ remain unchanged. We only need to compute the new last column of the tableau, i.e. the coefficients associated with the new Newton basis element $N_{n+1}(x)$. This takes only $O(n)$ work — exactly analogous to updating only the last diagonal in Neville's algorithm.